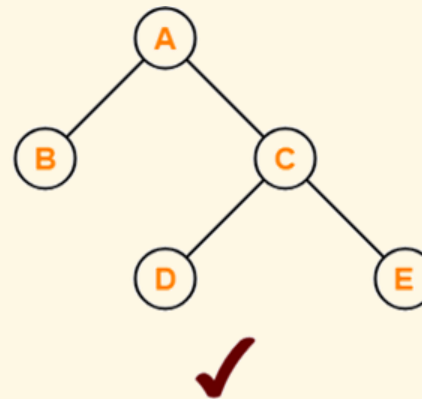
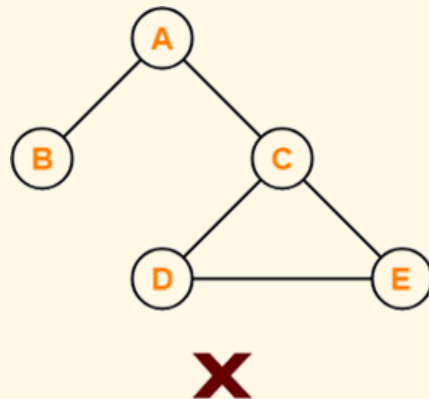
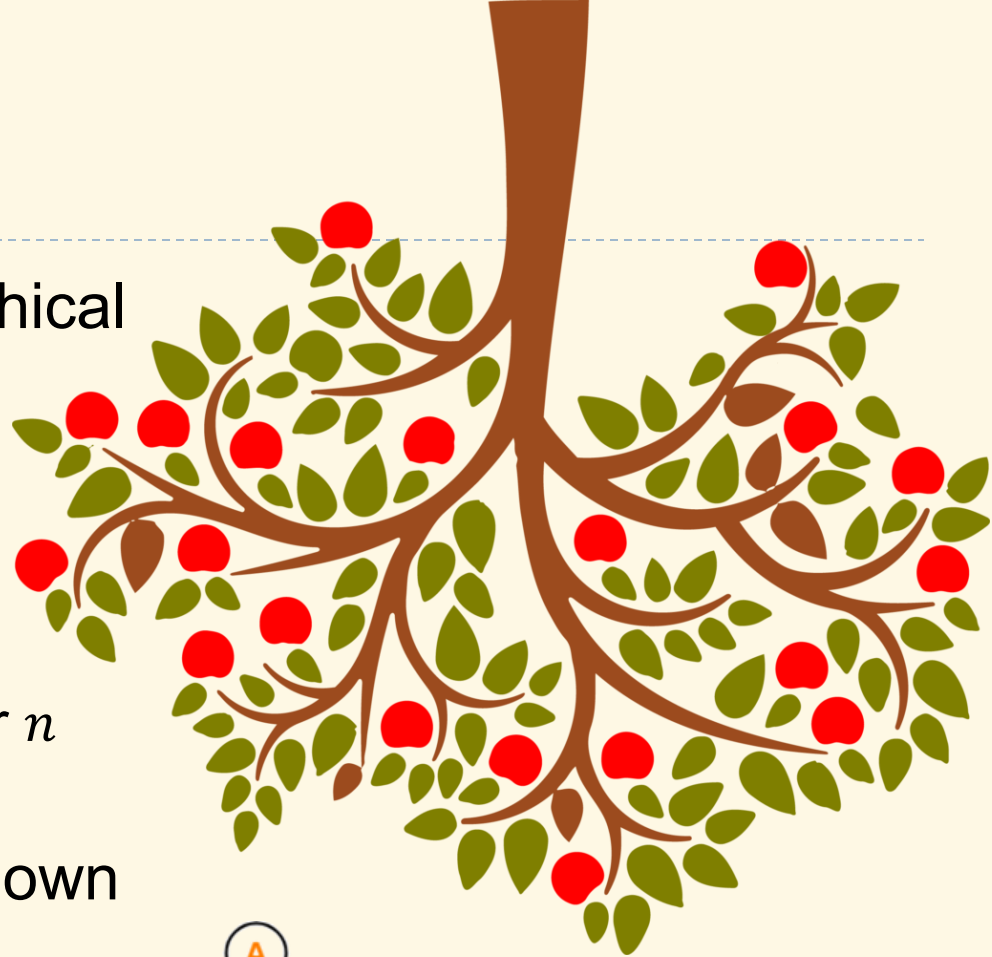

Trees

- ❑ General trees
- ❑ Binary trees
- ❑ Threaded binary trees
- ❑ Binary search trees
- ❑ Depth first & breadth first search

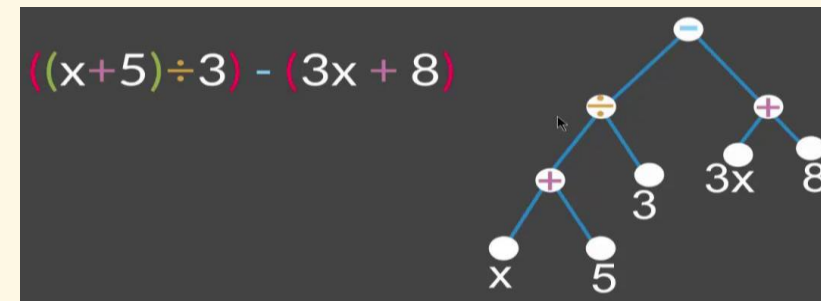
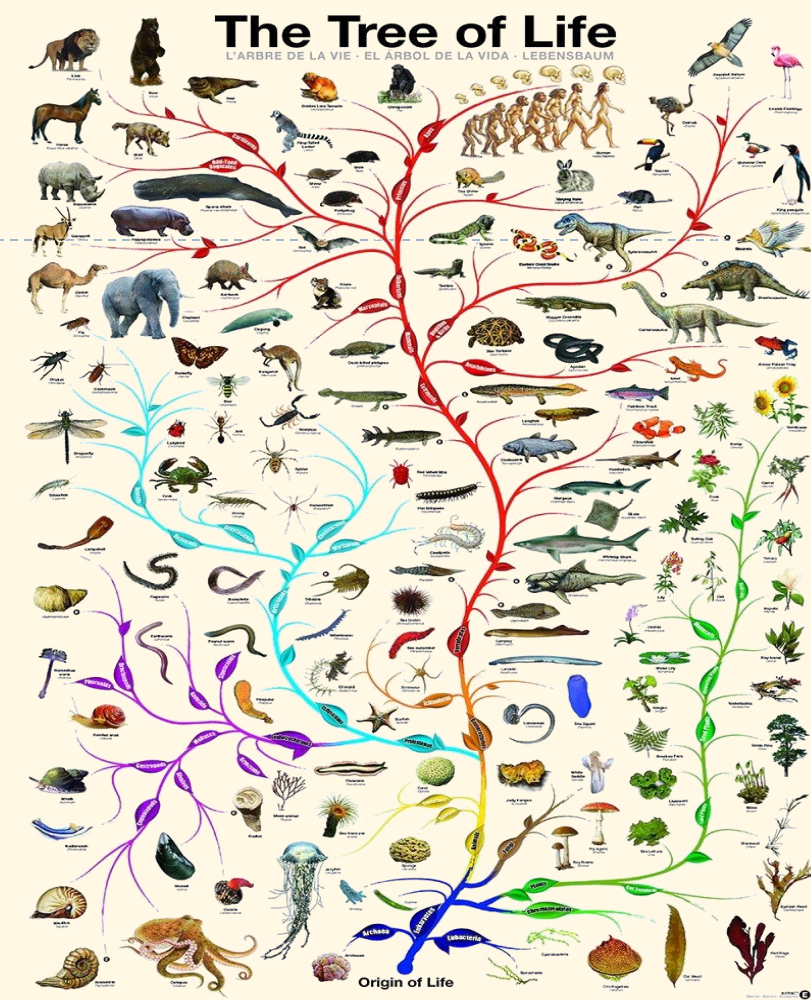
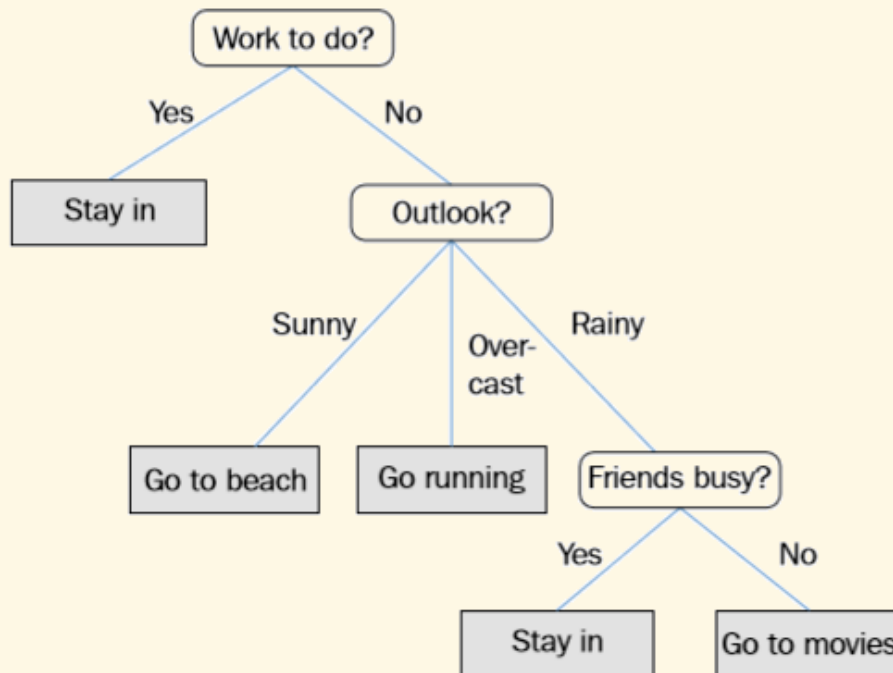
Trees

- ▶ Used to represent hierarchical data
- ▶ A special form of graph
 - ▶ Minimally connected
 - ▶ No loops, no circuits
 - ▶ Always has $n - 1$ edges for n vertices
- ▶ Usually depicted upside down

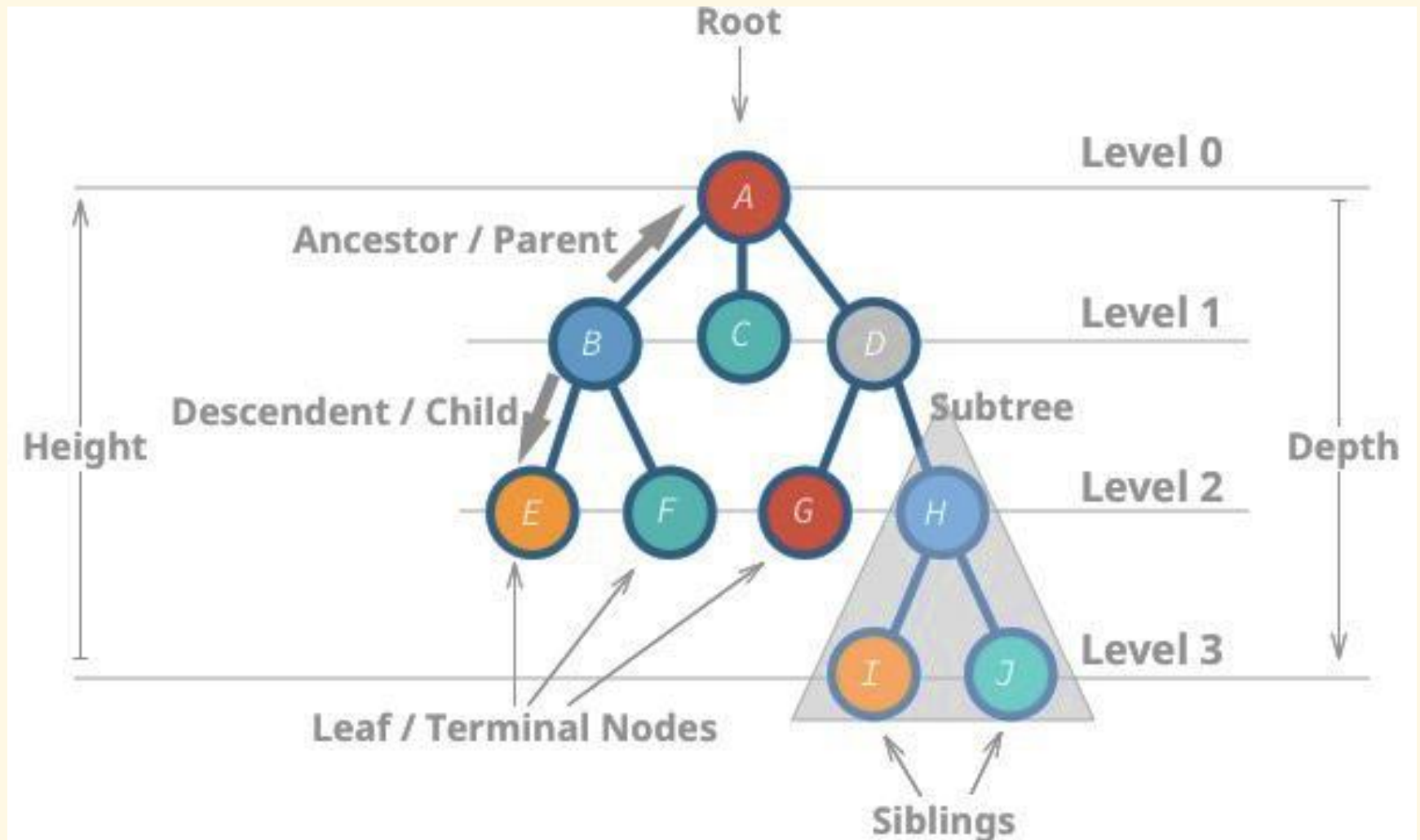


Application examples

- ▶ Classification
- ▶ Decision making
- ▶ Data storage, sorting, searching
- ▶ Computational methods



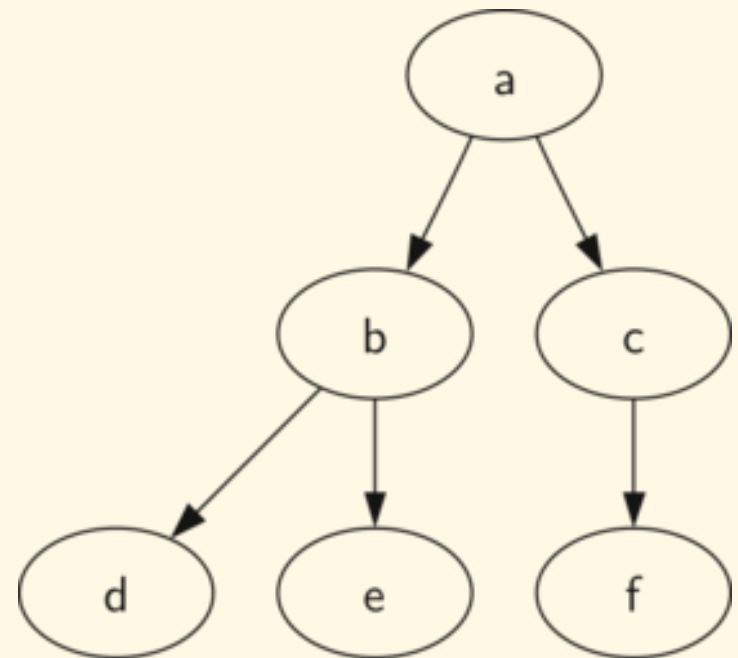
Terminology



List representation

- ▶ Each node has a list of zero or more child nodes

```
(a, [  
  (b, [  
    (d, ---),  
    (e, ---)  
  ]),  
  (c, [  
    (f, ---)  
  ])  
])
```



→ (a, [(b, [(d, ~~---~~) , (e, ~~---~~)]) , (c, [(f, ~~---~~)])])

Implementation

- ▶ Two approaches
 - ▶ Using an **array** to represent the children of each node
 - ▶ Using a **linked list** to represent the children of each node
- ▶ Possible to design in possessive or non-possessive manner
 - ▶ **Possessive**: a parent node takes over its child nodes, and is responsible to destroy them on its destruction
 - ▶ **Non-possessive**: a parent does not take over its child nodes, and leave them to destroy themselves

Exercises

1. Implement a simple general tree using an array (with possessive and non-possessive methods).
2. Implement a simple general tree using a linked list.
3. Write a function to save a tree to a string, and another to reconstruct the original tree from a string.

Binary tree

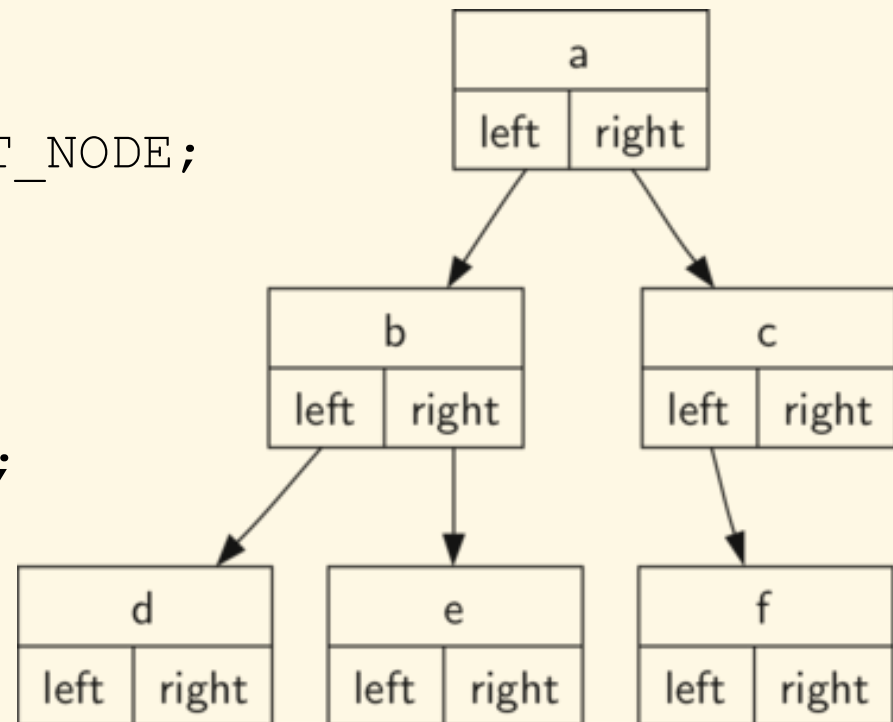
Definitions (with `int` data type)

- ▶ A tree in which each node can have at most 2 children: *left subtree* and *right subtree*

```
struct _BT_NODE;
```

```
typedef struct _BT_NODE BT_NODE;
```

```
struct _BT_NODE {  
    int data;  
    BT_NODE *left, *right;  
};
```



Basic operations

- ▶ `BT_NODE* btCreateNode(int v);`
- ▶ `BT_NODE* btCreateNodeLR(int v,
BT_NODE* l, BT_NODE* r);`
- ▶ `void btAddToLeft(BT_NODE* p, BT_NODE* c);`
- ▶ `void btAddToRight(BT_NODE* p, BT_NODE* c);`
- ▶ `void btDeleteLeft(BT_NODE* p);`
- ▶ `void btDeleteRight(BT_NODE* p);`
- ▶ `void btDeleteNode(BT_NODE** n);`

- ▶ `int btIsLeaf(BT_NODE* n);`

- ▶ `BT_NODE* btLeftMost(BT_NODE* n);`
- ▶ `BT_NODE* btRightMost(BT_NODE* n);`

Ways to construct a tree (1)

```
BT_NODE* a = btCreateNode(1);
```

```
BT_NODE* b = btCreateNode(2);
```

```
BT_NODE* c = btCreateNode(3);
```

```
BT_NODE* d = btCreateNode(4);
```

```
btAddToLeft(a, b);
```

```
btAddToRight(a, c);
```

```
btAddToRight(b, d);
```

Ways to construct a tree (2)

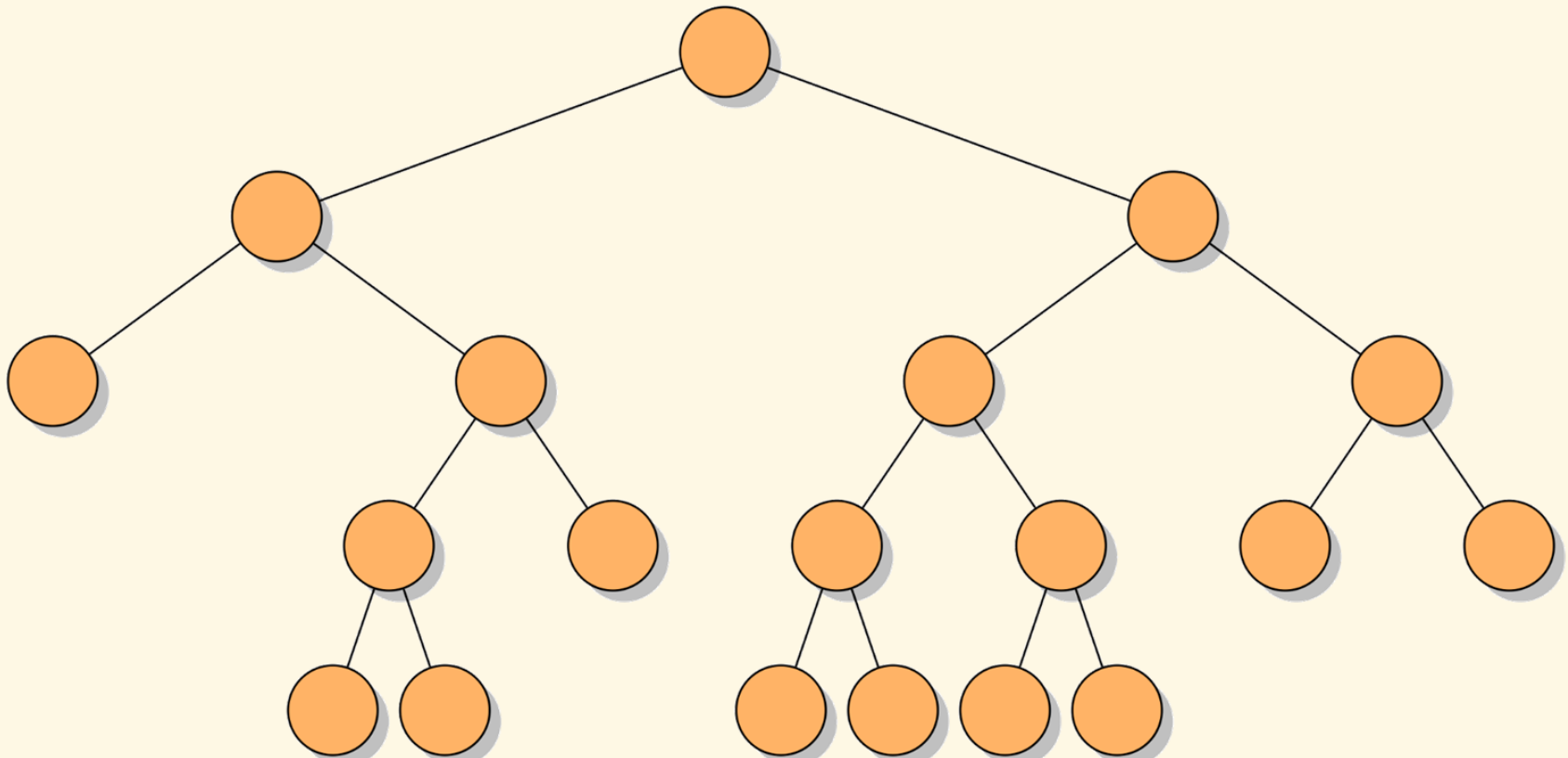
```
BT_NODE* a = btCreateNode(1);  
a->left = btCreateNode(2);  
a->right = btCreateNode(3);  
a->left->right = btCreateNode(4);
```

Ways to construct a tree (3)

```
BT_NODE* a =  
    btCreateNodeLR(1,  
        btCreateNodeLR(2,  
            NULL,  
            btCreateNodeLR(4, NULL, NULL)),  
        btCreateNodeLR(3, NULL, NULL));
```

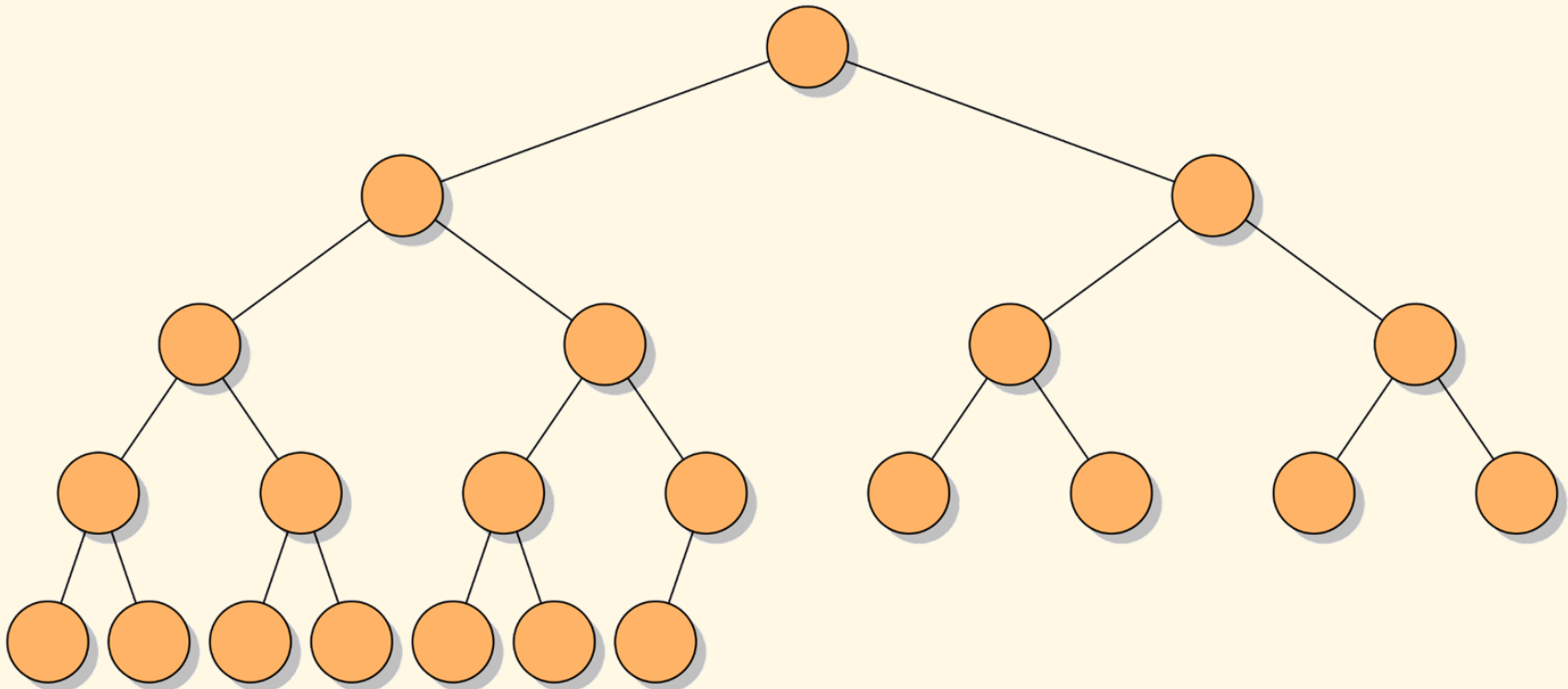
Full binary tree

- ▶ In a full binary tree, all nodes have either 0 or 2 children



Complete binary tree

- ▶ In a complete binary tree, every level, except possibly the last, is completely filled, and all nodes are as far left as possible



More operations

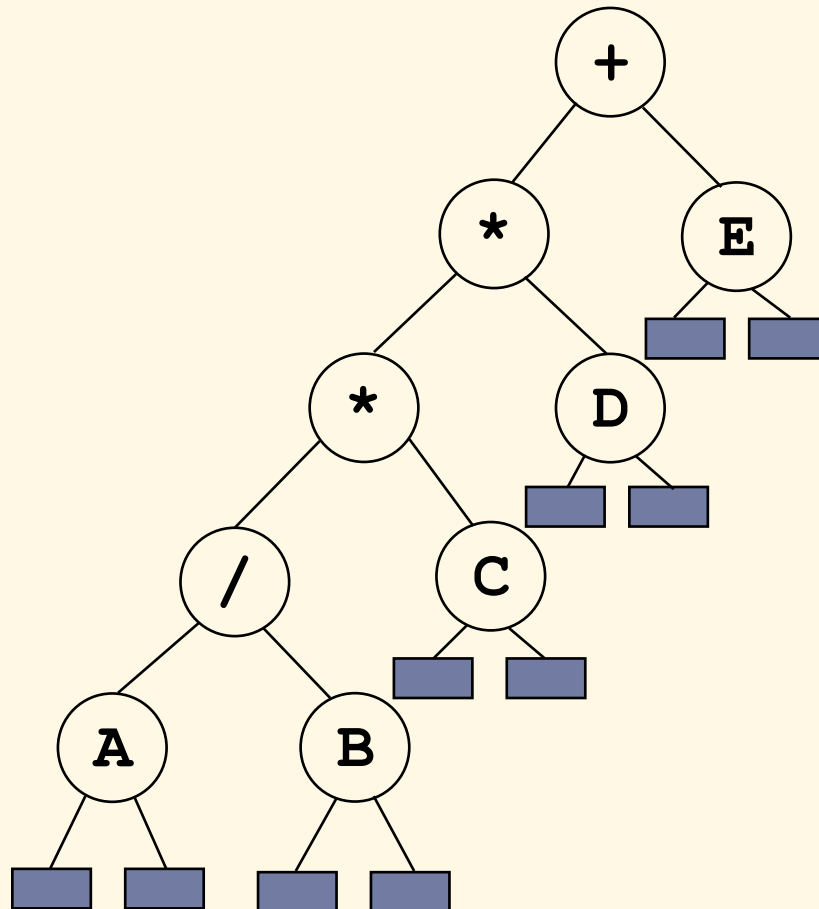
- ▶ `int btDepth(BT_NODE* n);`
- ▶ `int btCount(BT_NODE* n);`
- ▶ `int btIsFull(BT_NODE* n);`
- ▶ `int btIsComplete(BT_NODE* n);`

- ▶ `BT_NODE* btClone(BT_NODE* n);`
- ▶ `void btForEach(BT_NODE* n,
void (*callback)(int data, void* user),
void* user);`

- ▶ `void btPrintTree(BT_NODE* n);`
- ▶ `void btPrintList(BT_NODE* n);`

Binary tree traversals

► Arithmetic expression using binary tree



inorder traversal (LVR)

A / B * C * D + E

(infix expression)

preorder traversal (VLR)

+ * * / A B C D E

(prefix expression)

postorder traversal (LRV)

A B / C * D * E +

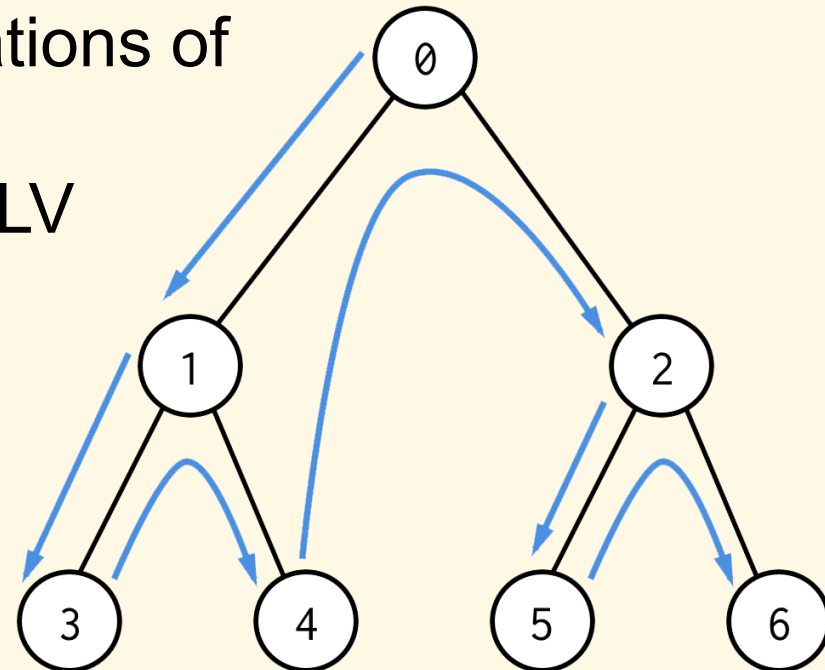
(postfix expression)

level order traversal

+ * E * D / C A B

Binary tree traversals: Depth first search (DFS)

- ▶ Let:
 - ▶ L: moving left
 - ▶ R: moving right
 - ▶ V: visiting node
- ▶ There are 6 possible combinations of traversal:
LVR, LRV, VLR, VRL, RVL, RLV
- ▶ Two ways of implementation
 - ▶ Recursive
 - ▶ Iterative (using a stack)



DFS implementation using recursion (VLR)

```
void btDepthFirstSearch(BT_NODE* n) {  
    if (n == NULL) return;  
  
    // do something with n->data...  
  
    if (n->left != NULL)  
        btTraverseDFSRecursive(n->left);  
  
    if (n->right != NULL)  
        btTraverseDFSRecursive(n->right);  
}
```

DFS implementation using recursion (LVR)

```
void btDepthFirstSearch(BT_NODE* n) {  
    if (n == NULL) return;  
  
    if (n->left != NULL)  
        btTraverseDFSRecursive(n->left);  
  
    // do something with n->data...  
  
    if (n->right != NULL)  
        btTraverseDFSRecursive(n->right);  
}
```

DFS implementation using recursion (LRV)

```
void btDepthFirstSearch(BT_NODE* n) {  
    if (n == NULL) return;  
  
    if (n->left != NULL)  
        btTraverseDFSRecursive(n->left);  
  
    if (n->right != NULL)  
        btTraverseDFSRecursive(n->right);  
  
    // do something with n->data...  
}
```

DFS implementation using stack (VLR)

```
void btDepthFirstSearch(BT_NODE* n) {  
    STACK stack;  
    init(&stack);  
    push(&stack, n);  
  
    while (!empty(stack)) {  
        n = top(stack);  
        pop(&stack);  
  
        printf("%d ", n->data);  
  
        if (n->right != NULL) push(&stack, n->right);  
        if (n->left != NULL) push(&stack, n->left);  
    }  
}
```

DFS implementation using stack (LVR)

```
void btDepthFirstSearch(BT_NODE* n) {  
    STACK stack;  
    init(&stack);  
  
    for (;;) {  
        for (; n != NULL; n = n->left) push(&stack, n);  
        if (empty(stack)) break;  
  
        n = top(stack);  
        pop(&stack);  
        printf("%d ", n->data);  
  
        n = n->right;  
    }  
}
```

DFS implementation using stack (LRV)

```
void btDepthFirstSearch(BT_NODE* n) {
    STACK stack;
    init(&stack);

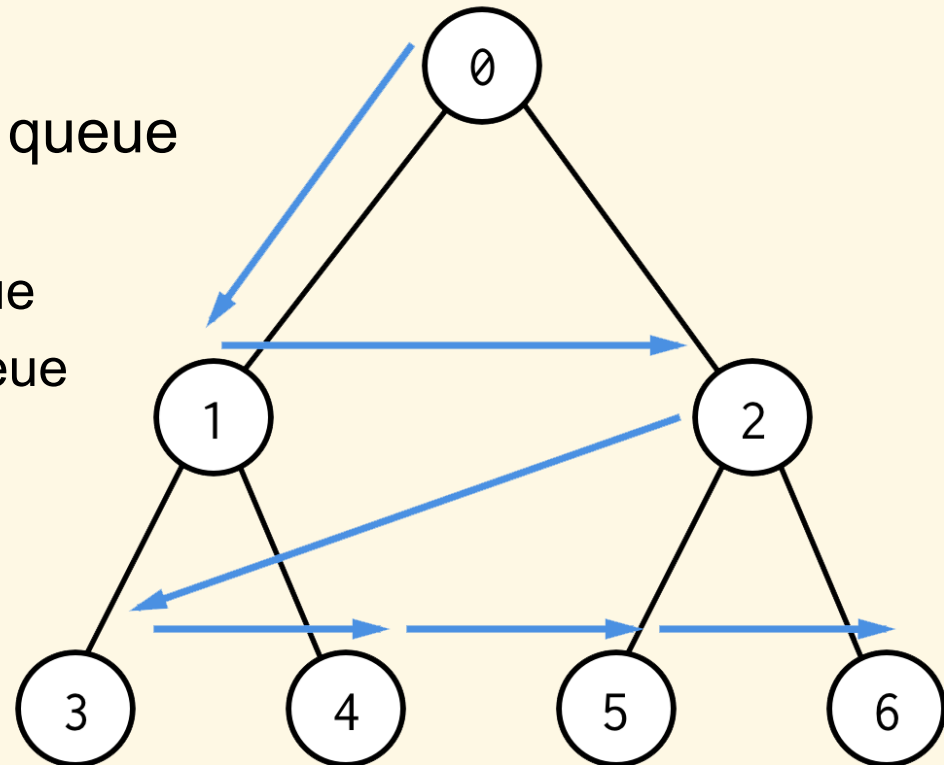
    for (;;) {
        for (; n != NULL; n = n->left) {
            push(&stack, n);
            push(&stack, n);
        }
        if (empty(stack)) break;

        n = top(stack);
        pop(&stack);

        if (!empty(stack) && top(stack) == n) n = n->right;
        else {
            printf("%d ", n->data);
            n = NULL;
        }
    }
}
```


Binary tree traversals: Breadth first search (BFS)

- ▶ Go across to siblings before visiting all nodes on a given level in left-to-right order
- ▶ Implementation (using a queue):
 - ▶ Add root node to queue
 - ▶ For a given node from the queue
 - ▶ Visit node
 - ▶ Add nodes left child to queue
 - ▶ Add nodes right child to queue



BFS implementation using queue

```
void btBreadthFirstSearch(BT_NODE* n) {
    QUEUE queue;
    init(&queue);
    enqueue(&queue, n);

    while (!empty(queue)) {
        n = top(queue);
        dequeue(&queue);

        printf("%d ", n->data);

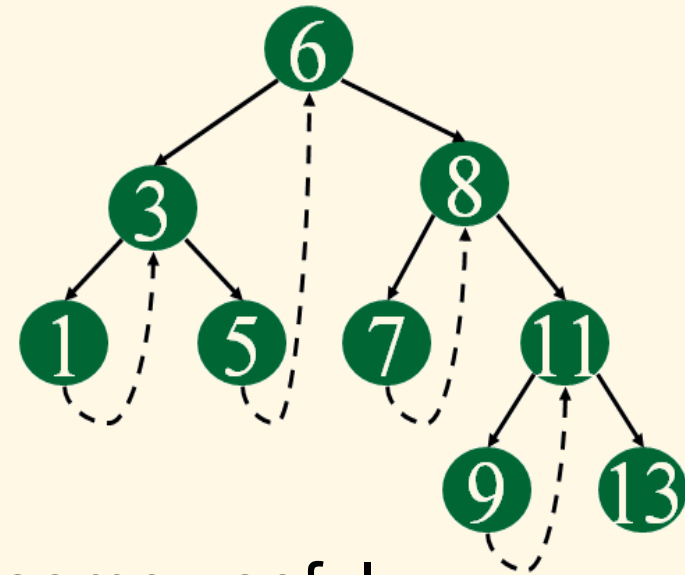
        if (n->left != NULL) enqueue(&queue, n->left);
        if (n->right != NULL) enqueue(&queue, n->right);
    }
}
```

Threaded binary tree (TBT)

Overview

- ▶ Two many null pointers in current representation of binary trees (waste of memory)

- ▶ n : number of nodes
- ▶ number of non-null links: $n - 1$
- ▶ total links: $2n$
- ▶ null links: $2n - (n - 1) = n + 1$



➔ Replace these null pointers with some useful “threads”

- ▶ NULL right pointers is made to point to the in-order successor (*single right threaded*)

Representation

```
struct _TBT_NODE {  
    int data;  
    TBT_NODE *left, *right;  
    int isRightThread;  
}
```

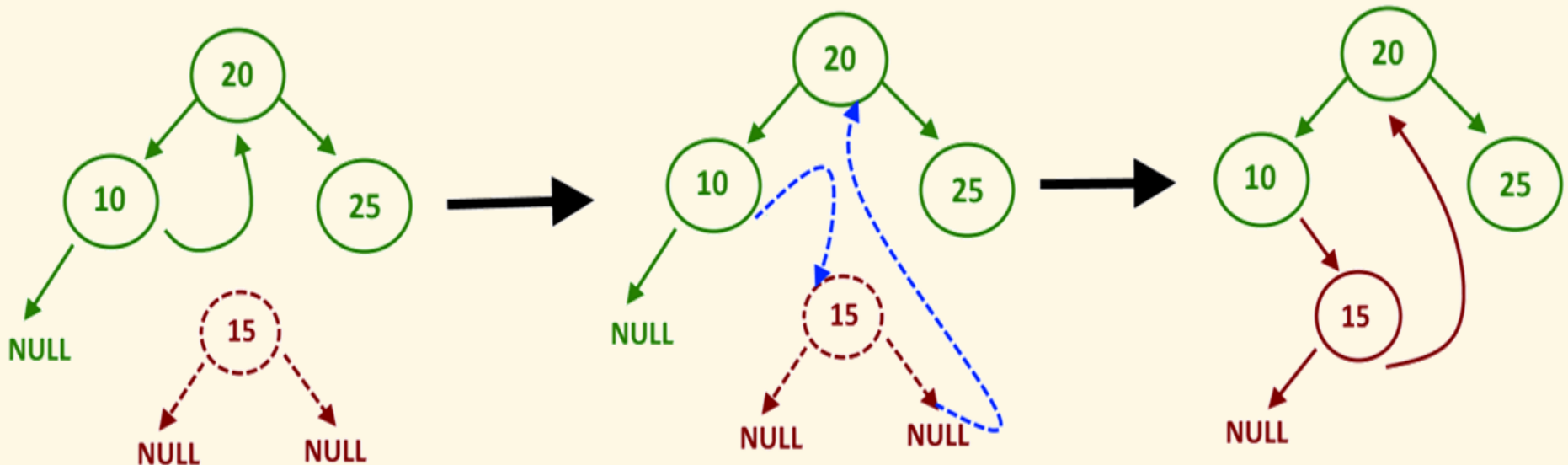
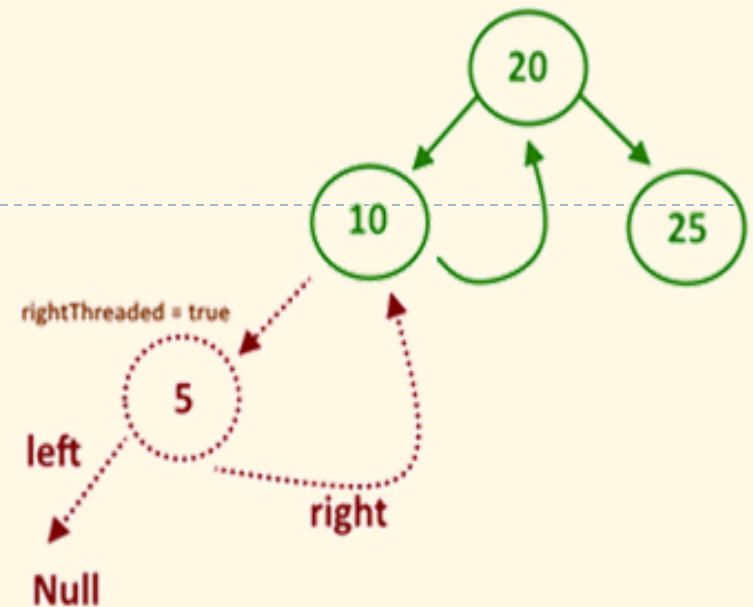
DFS traversal

► No more stack neither recursion

```
void tbtTraverse(BT_NODE* n) {  
    BT_NODE* cur = btLeftMost(n);  
  
    while (cur != NULL) {  
        printf("%d ", cur->data);  
  
        if (cur->isRightThread)  
            cur = cur->right;  
        else  
            cur = btLeftMost(cur->right);  
    }  
}
```

Insertion

- ▶ 2 cases
 - ▶ To the left of parent node
 - ▶ To the right of parent node

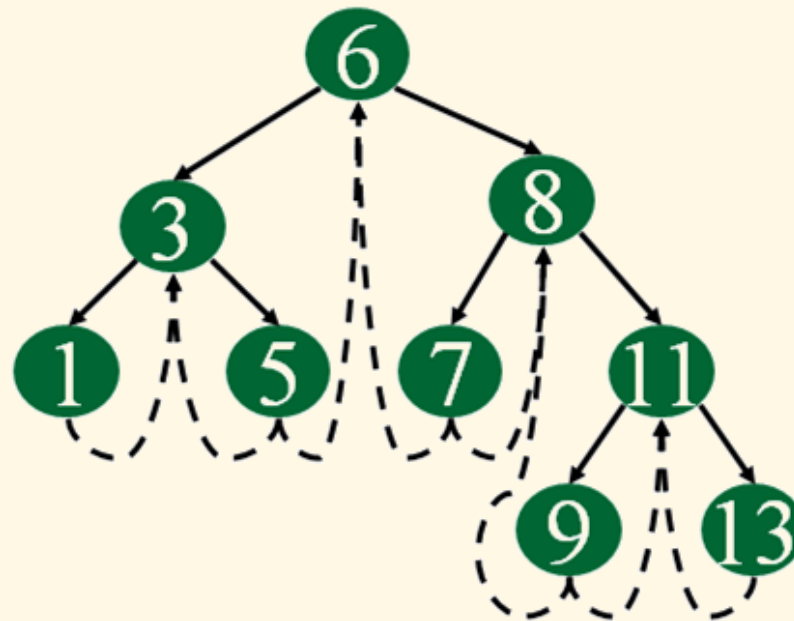


Deletion

- ▶ Also 2 cases
 - ▶ Delete left subtree
 - ▶ Delete right subtree

Double threaded binary tree

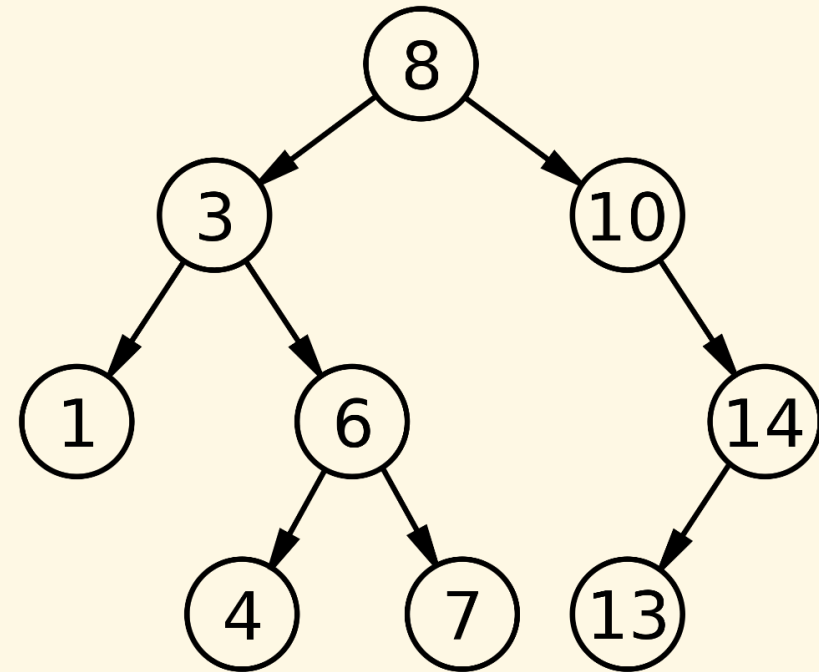
- ▶ Both left and right NULL pointers are made to point to in-order predecessor and in-order successor, respectively
- ▶ The predecessor threads are useful for reverse in-order traversal and post-order traversal



Binary search tree (BST)

Overview

- ▶ Binary tree can be used in binary search
- ▶ However, the nodes need to be organized in a specific order: for every node,
 - ▶ the values on left branch must be smaller than its one
 - ▶ the values on the right branch must be greater its one
 - ▶ there are not two nodes with the same key
- ▶ Note that subtrees are also BST



Operations (with `int` data type)

- ▶ `void bstInsertNode (BT_NODE** n, int x);`
- ▶ `void bstDeleteNode (BT_NODE** n, int x);`
- ▶ `BT_NODE* bstSearch (BT_NODE* n, int x);`

Insert node in BST

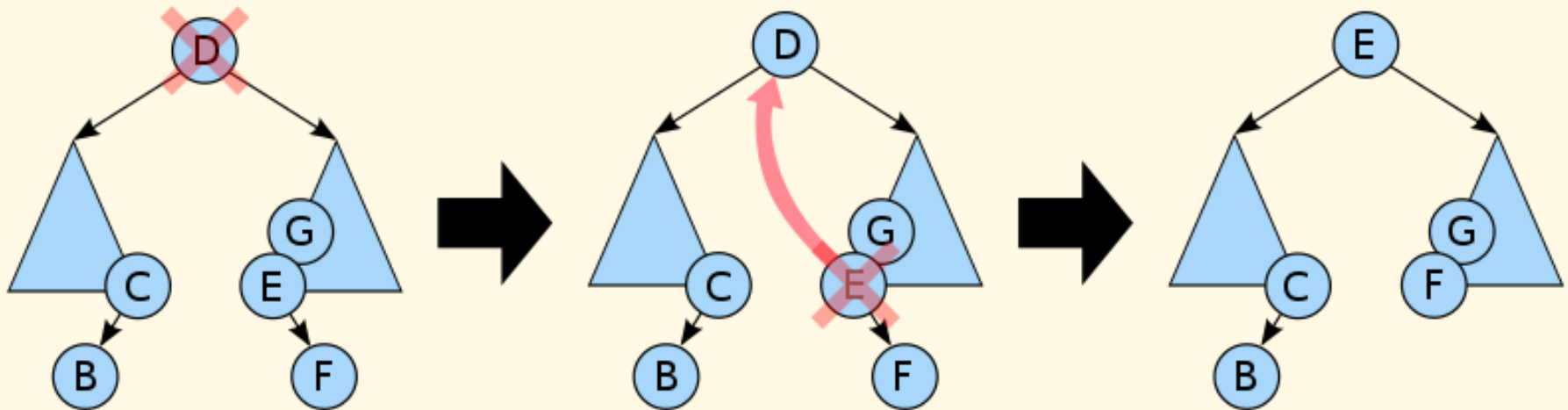
```
void bstInsertNode(BT_NODE** n, int x) {
    if (*n == NULL) {
        *n = (BT_NODE*)malloc(sizeof(BT_NODE));
        (*n)->data = x;
        (*n)->left = NULL;
        (*n)->right = NULL;
    } else if (x < (*n)->data)
        bstInsertNode(&(*n)->left, x);
    else if (x > (*n)->data)
        bstInsertNode(&(*n)->right, x);
}
```

Search node in BST

```
BT_NODE* bstSearch(BT_NODE* n, int x) {  
    if (n == NULL) return NULL; // not found  
    if (n->data == x) return n; // found  
  
    if (n->data > x) return bstSearch(n->left, x);  
    return bstSearch(n->right, x);  
}
```

Delete node in BST

- ▶ 3 cases
 - ▶ Delete a node with no children
 - ▶ Delete a node with one child
 - ▶ Delete a node with two children (see *picture*)



BST shapes

- ▶ The order of supplying the data determines where it is placed in the BST, which determines the shape of the BST
- ▶ Create BSTs from the same set of data presented each time in a different order:
 - ▶ 17 4 14 19 15 7 9 3 16 10
 - ▶ 9 10 17 4 3 7 14 16 15 19
 - ▶ 19 17 16 15 14 10 9 7 4 3
- ➔ Balanced search trees to avoid tree degeneration
 - ▶ AVL trees
 - ▶ 2-3-4 trees
 - ▶ Red-black trees

AVL tree *(for reference)*

Overview

- ▶ Operations on binary search tree are $O(h)$ - where h is tree height
 - ▶ Tree must be balanced so that $h \approx \log n$
- ▶ Add and remove operations do not necessarily maintain that balance
- ▶ We seek ways to maintain balance and thus maintain efficiency

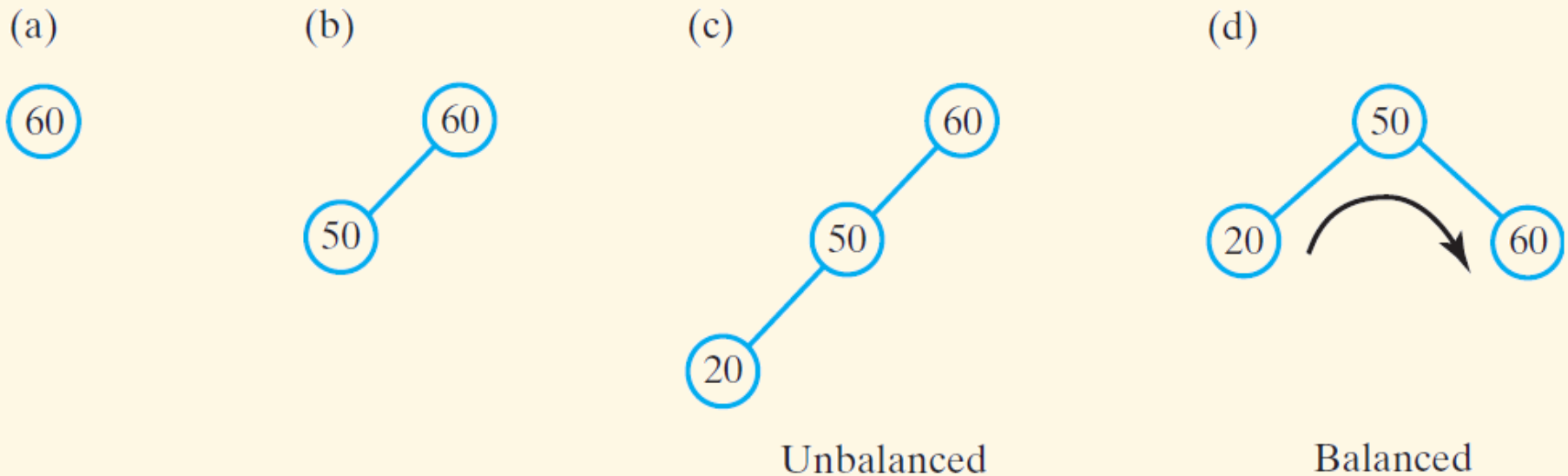
AVL tree

- ▶ Binary search tree that rearranges nodes whenever it becomes unbalanced
 - ▶ Happens during addition or removal of nodes
- ▶ Uses
 - ▶ Left rotation
 - ▶ Right rotation
 - ▶ Balanced node – the root of a balanced tree



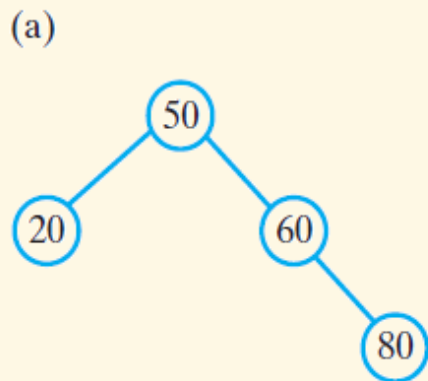
AVL tree

- ▶ (a, b, c) After inserting 60, 50 and 20 into an initially empty binary search tree, the tree is not balanced
- ▶ (d) A corresponding AVL tree rotates its nodes to restore balance

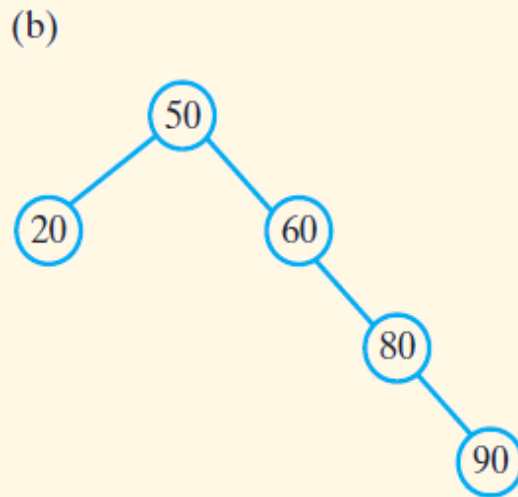


AVL tree

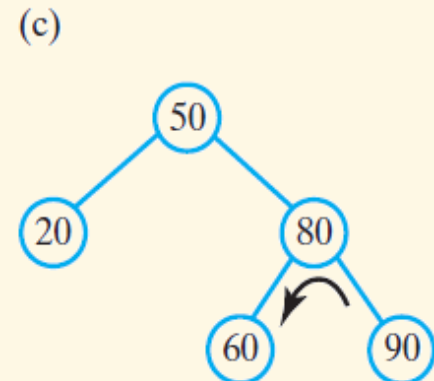
- ▶ (a) Adding 80 to the tree in Figure 27-1d does not change the balance of the tree
- ▶ (b) A subsequent addition of 90 makes the tree unbalanced
- ▶ (c) A left rotation restores its balance



Balanced



Unbalanced

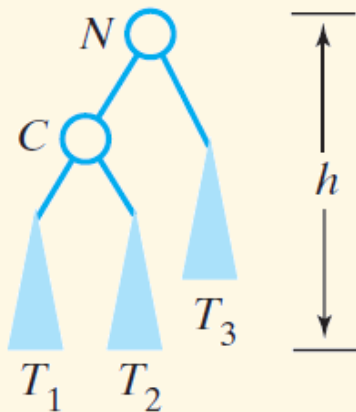


Balanced

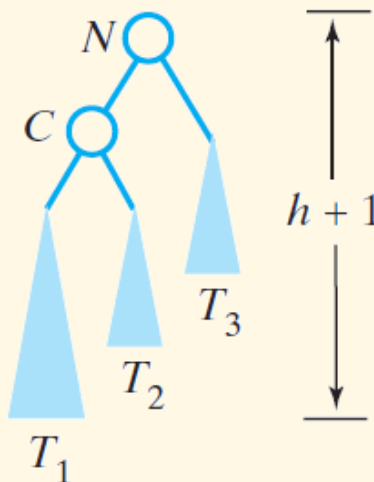
AVL tree

- Before and after an addition to an AVL subtree that requires a right rotation to maintain its balance

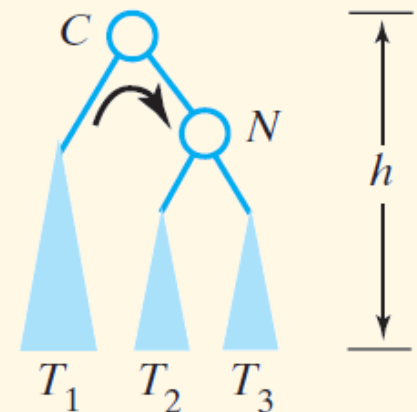
(a) Before addition



(b) After addition

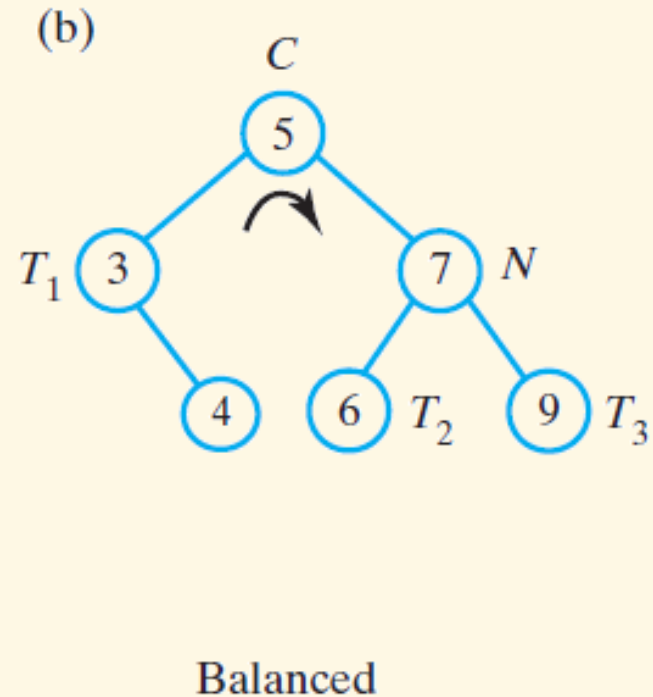
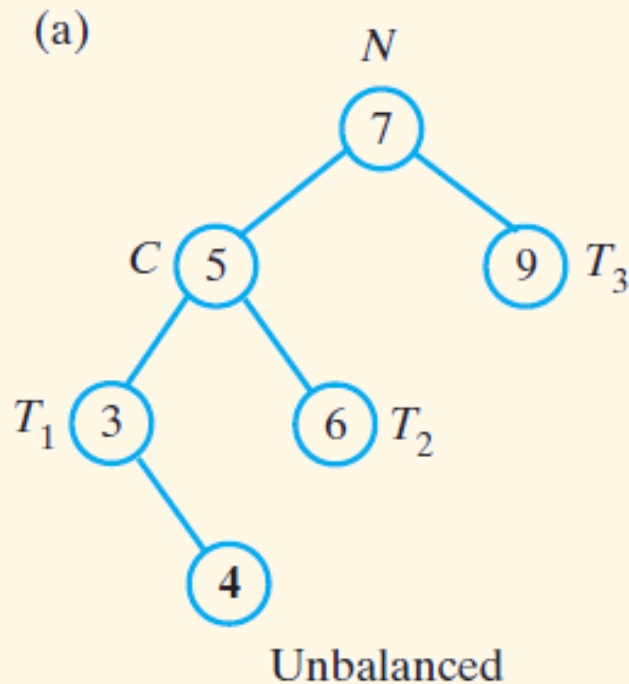


(c) After right rotation



AVL tree

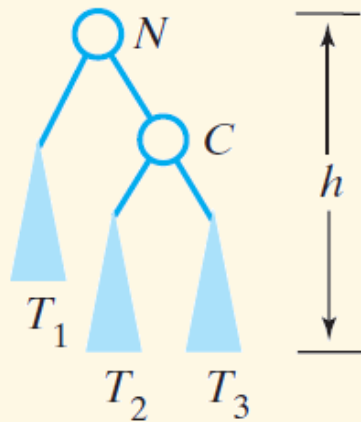
- Before and after a right rotation restores balance to an AVL tree



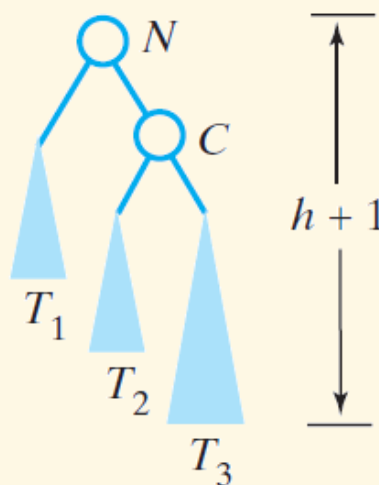
AVL tree

- Before and after an addition to an AVL subtree that requires a left rotation to maintain its balance

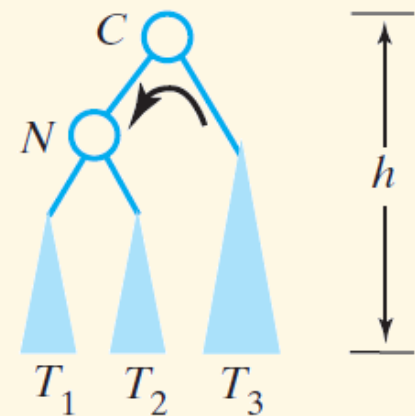
(a) Before addition



(b) After addition



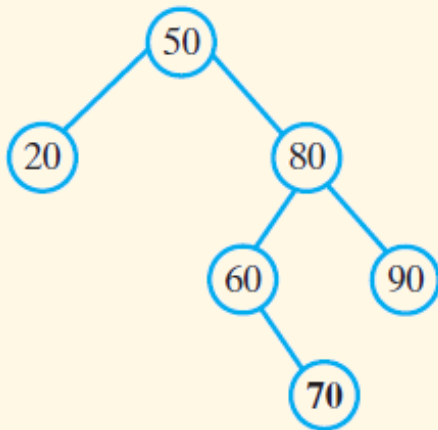
(c) After left rotation



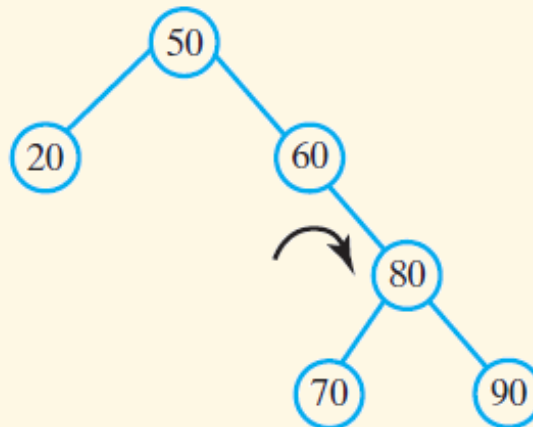
AVL tree

- ▶ (a) Adding 70 to the tree destroys its balance
- ▶ (b, c) To restore the balance, perform both a right rotation and a left rotation

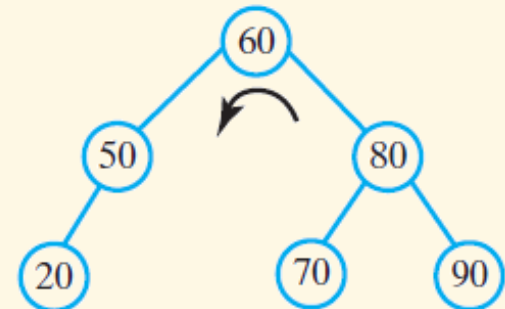
(a) After adding 70



(b) After right rotation



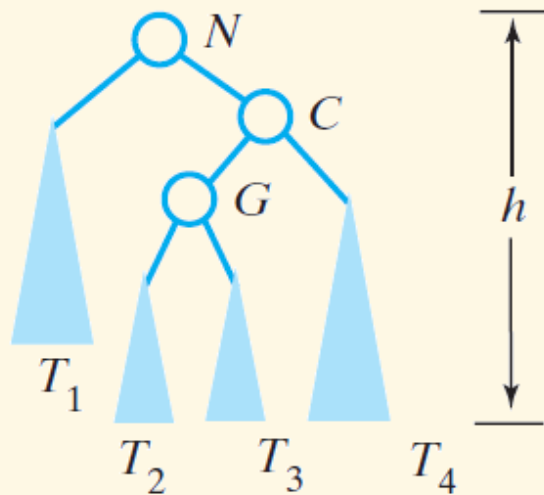
(c) After left rotation



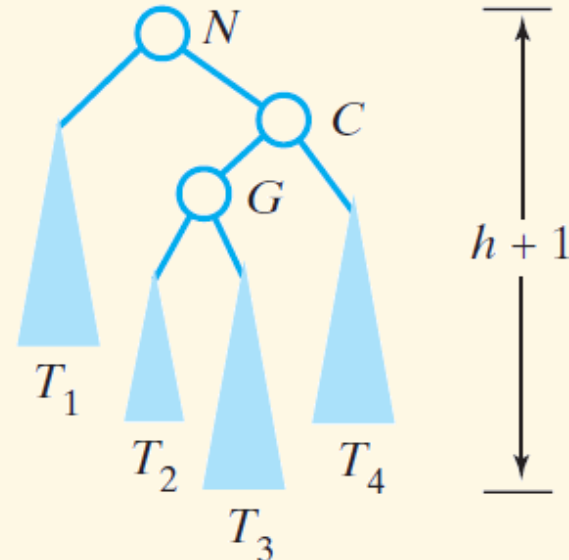
AVL tree

- Before and after an addition to an AVL subtree that requires both a right rotation and a left rotation to maintain its balance

(a) Before addition



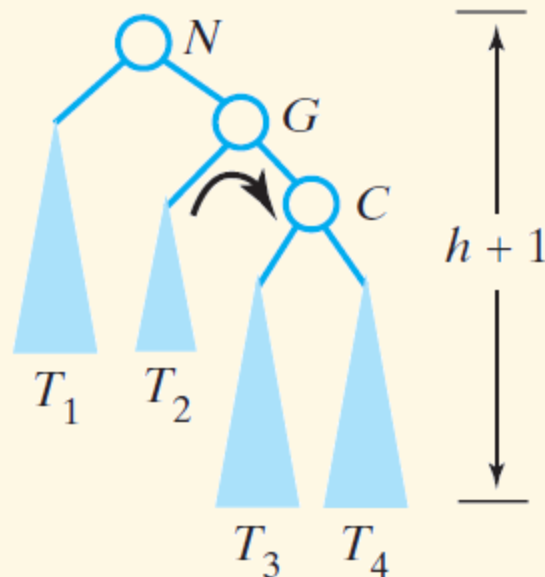
(b) After addition



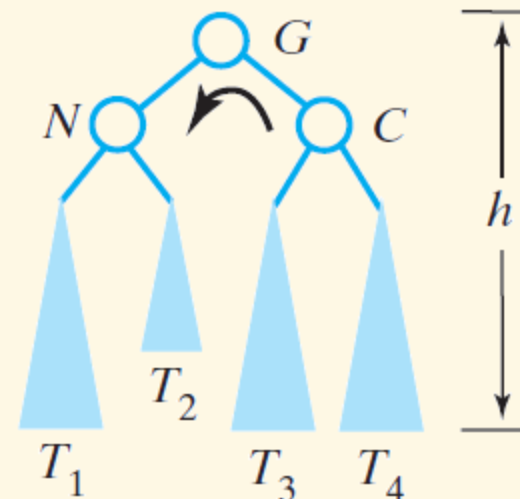
AVL tree

- Before and after an addition to an AVL subtree that requires both a right rotation and a left rotation to maintain its balance

(c) After right rotation



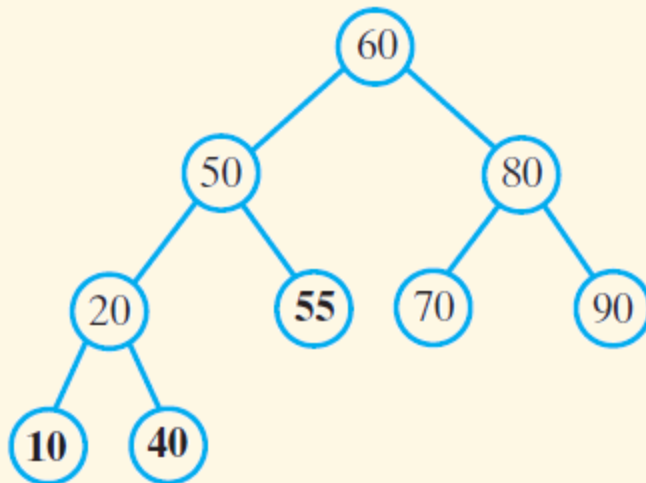
(d) After left rotation



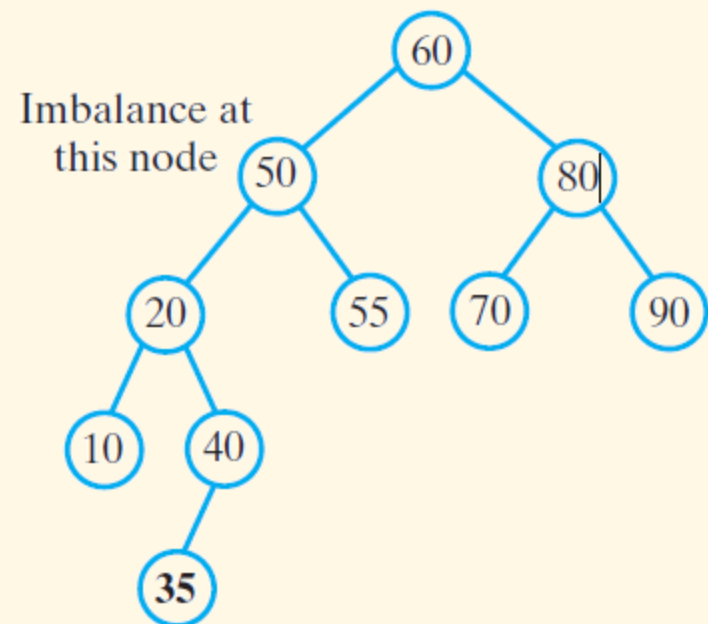
AVL tree

- ▶ (a) After additions that maintain its balance
- ▶ (b) After an addition that destroys the balance

(a) After adding 55, 10, and 40



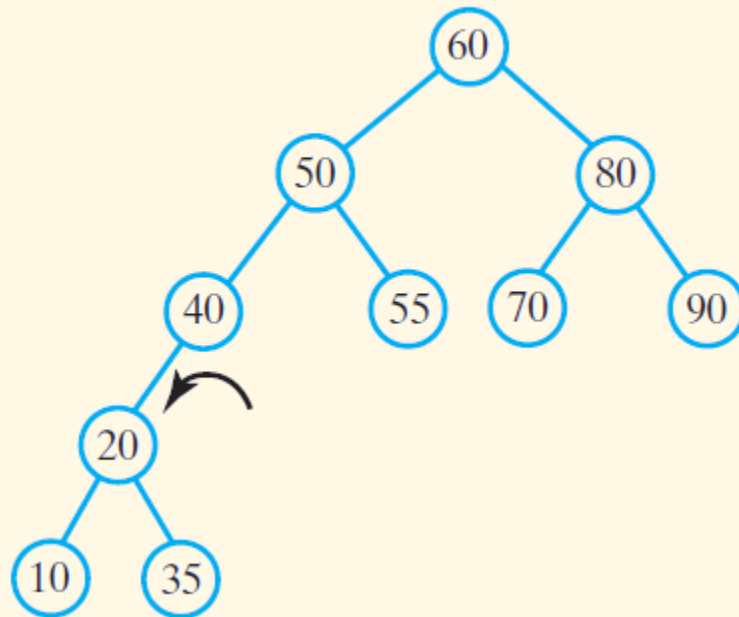
(b) After adding 35



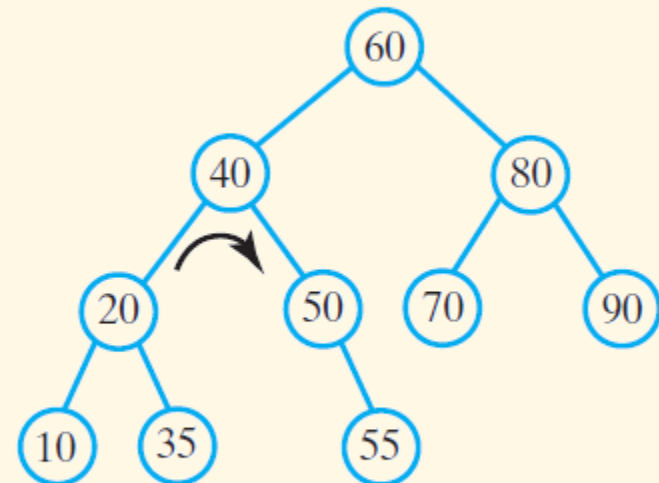
AVL tree

- ▶ (c) after a left rotation
- ▶ (d) after a right rotation

(c) After left rotation about 40



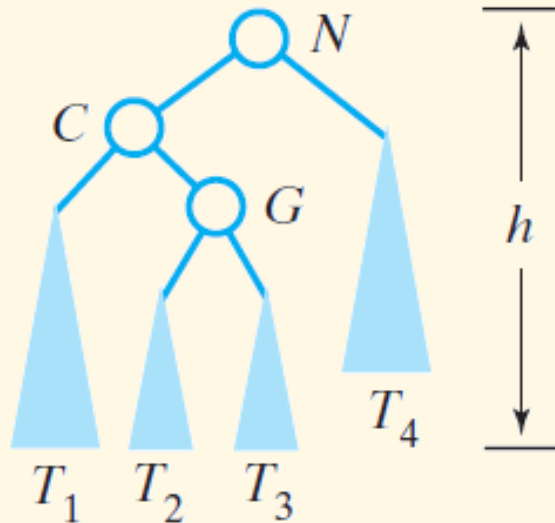
(d) After right rotation about 40



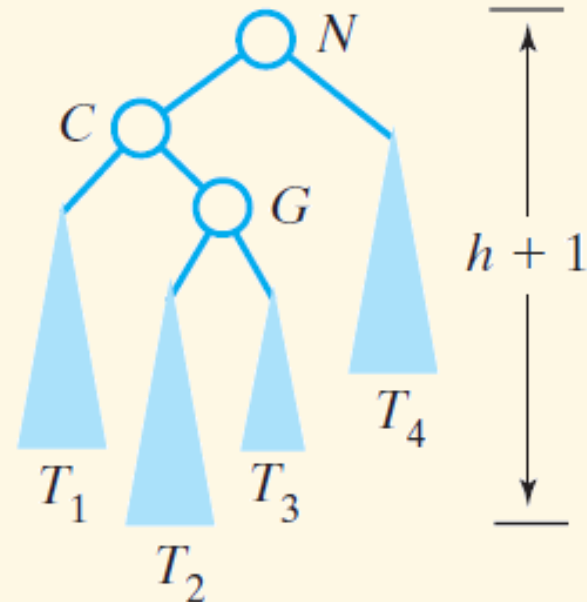
AVL tree

- Before and after an addition to an AVL subtree that requires both a left rotation and a right rotation to maintain its balance

(a) Before addition



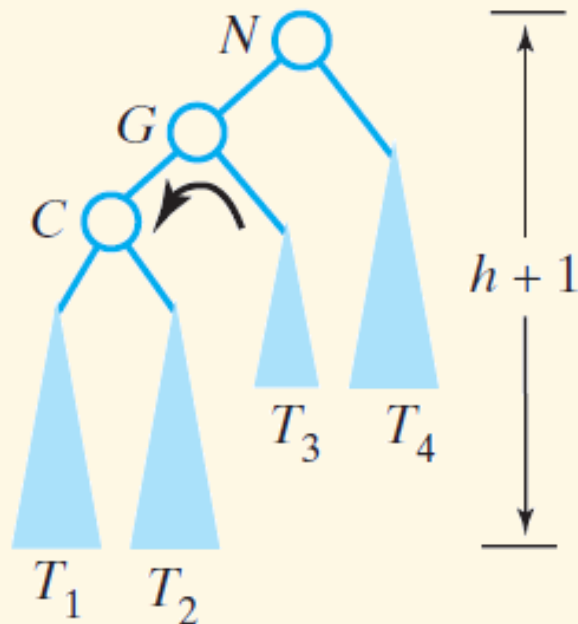
(b) After addition



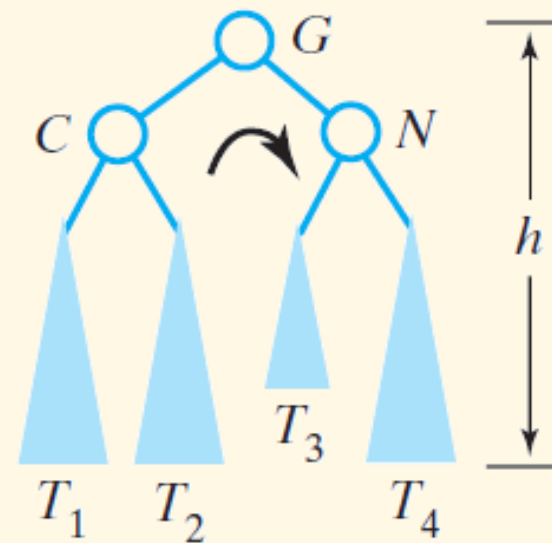
AVL tree

- Before and after an addition to an AVL subtree that requires both a left rotation and a right rotation to maintain its balance

(c) After left rotation



(d) After right rotation



Homework

1. Write a function to print a binary tree in a pretty way (see picture)
2. Write a function to calculate number of leaf nodes in a tree
3. Write a function to check if a binary tree is complete
4. Write a function to check if a binary tree is BST
5. Implement the deletion of nodes in BST
6. Write a program to create a binary tree representing the following expression, then evaluate the value using the tree

$$25^2/5 - (6 + 3)^2 \times (4 + 2)$$

